

Fatiamento, tuplas e dicionários

Capítulo 3

Fatiamento (slicing) de listas e strings

Capítulo 4

Tuplas e dicionários (e seu uso em funções)

Fatiamento de listas

- A sintaxe `lista[início:fim:passo]`
- A regra que confunde todo mundo: o fim é EXCLUÍDO
- Passo, passo negativo, substituição e remoção

A sintaxe do fatiamento

Três parâmetros, todos opcionais

```
lista[início : fim : passo]
```

início

Índice do primeiro elemento
INCLUÍDO

padrão: 0

fim

Índice do primeiro elemento
EXCLUÍDO

padrão: tamanho da lista

passo

De quanto em quanto andar

padrão: 1

A regra que confunde todo mundo

numeros[2:7] → começa no 2, para ANTES do 7

início = 2
entra na fatia

fim = 7
NÃO entra



```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(numeros[2:7])
[2, 3, 4, 5, 6]
```

Truque para lembrar

O resultado tem fim – início itens.
 $7 - 2 = 5$ elementos.

Consequência prática: `numeros[:3] + numeros[3:]` reconstrói a lista inteira, sem faltar nem repetir nenhum elemento.

Omitindo parâmetros

Cada omissão vira o valor padrão — não um erro

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numeros[:4])    # sem início → começa do 0
[0, 1, 2, 3]

print(numeros[6:])    # sem fim → vai até o final
[6, 7, 8, 9]

print(numeros[:])     # sem nada → cópia da lista
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

[:] é cópia, não apelido

`copia = orig[:]` cria uma lista nova.
`alias = orig` aponta para a MESMA lista.

Fatiar não estoura

`numeros[2:100]` devolve o que existe.
`numeros[100]` levanta `IndexError`.

O terceiro parâmetro: passo

Salta elementos — e, se for negativo, anda para trás

```
print(numeros[::2])      # de 2 em 2
[0, 2, 4, 6, 8]
print(numeros[1::2])     # ímpares
[1, 3, 5, 7, 9]
print(numeros[::-1])     # inverte
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Armadilha do passo negativo

`numeros[2:7:-1]` devolve [] — lista vazia.

Andando para trás, para sair do 2 e chegar ao 7 não há caminho. O correto seria `numeros[7:2:-1]`.

Por que usar isso?

- Legibilidade: a intenção fica na própria linha
- Eficiência: opera direto na lista, sem loop explícito
- Flexibilidade: extrair, substituir e remover com a mesma sintaxe

Fatia também escreve e apaga

O fatiamento não é só leitura

Substituir um trecho

```
numeros = [0,1,2,3,4,5,6,7,8,9]
numeros[2:5] = [20, 30, 40]
print(numeros)
[0, 1, 20, 30, 40, 5, 6, 7, 8, 9]
```

Remover um trecho

```
numeros = [0,1,2,3,4,5,6,7,8,9]
del numeros[2:5]
print(numeros)
[0, 1, 5, 6, 7, 8, 9]
```

Os tamanhos não precisam bater

`numeros[2:5] = [99]` troca 3 elementos por 1 e a lista encolhe: `[0, 1, 99, 5, 6, 7, 8, 9]`. O interpretador reacomoda tudo — diferente da atribuição por índice, que troca exatamente um item.

1

PRÁTICA 1 · 5 minutos

Recortando uma lista de notas

Arquivo: `pratica_01.py`

```
notas = [7.5, 6.0, 9.0, 8.5, 5.5, 10.0, 4.0, 8.0]
```

1. Imprima as 3 primeiras notas
2. Imprima as 2 últimas notas
3. Imprima a lista invertida
4. Imprima uma nota sim, uma não
5. Troque os índices 2 a 4 por [9.9, 9.9, 9.9]

Saídas esperadas

```
[7.5, 6.0, 9.0]
```

```
[4.0, 8.0]
```

```
[8.0, 4.0, 10.0, 5.5, 8.5, 9.0, 6.0, 7.5]
```

```
[7.5, 9.0, 5.5, 4.0]
```

```
[7.5, 6.0, 9.9, 9.9, 9.9, 10.0, 4.0, 8.0]
```

Terminou antes? Descubra o que faz `notas[::-2]` e explique em uma frase.

Operações com strings

- Texto em Python é uma sequência — vale tudo que você acabou de aprender
- Concatenação, repetição, len(), índice e in
- A diferença crucial: string é IMUTÁVEL

Operações básicas com strings

Os operadores que você já conhece, aplicados a texto

+ concatena

```
s1 = "Olá"  
s2 = "Mundo"  
print(s1 + " " + s2)
```

saída: Olá Mundo

* repete

```
s = "Python "  
print(s * 3)
```

saída: Python Python Python

len() conta

```
s = "Python"  
print(len(s))
```

saída: 6

Detalhe fino: "Python " tem um espaço no fim. Por isso a repetição sai separada — e sobra um espaço invisível no final da saída.

Fatiando texto

Mesma sintaxe das listas — `s = "Python"`

```
s = "Python"

print(s[0])      P
print(s[-1])     n
print(s[0:4])    Pyth
print(s[2:])     thon
print(s[-4:])    thon
print(s[:2])     Pto
print(s[::-1])   nohtyP
```

String é imutável

`s[0] = "J"` não funciona:
TypeError: 'str' object does
not support item assignment

Verificando conteúdo

```
frase = "Python é incrível"
print("Python" in frase)      True
print("Java" not in frase)    True
```

2

PRÁTICA 2 · 4 minutos

Detector de palíndromo

Arquivo: `pratica_02.py`

1. Guarde a palavra "arara" em uma variável
2. Compare a palavra com ela invertida
3. Imprima True ou False
4. Repita com "Python"
5. Bônus: faça funcionar com a frase abaixo

Pistas

Inverter é `[::-1]`

Comparar é `==`

Frase do bônus:

"Anotaram a data da maratona"

Precisa de `.lower()` e

`.replace(" ", "")`

Saídas: True para arara · False para Python · True para a frase do bônus

Tuplas

- Quando a ORDEM dos dados é parte do significado
- Quando o dado NÃO pode mudar depois de criado
- Desempacotamento: a forma elegante de ler uma tupla

Tupla: ordem e imutabilidade

Parênteses em vez de colchetes — e o conteúdo fica congelado

```
# coordenada geográfica (lat, long)
lat_long_senac = (-23.668559, -46.702251)

vermelho_rgb = (255, 0, 0)
amarelo_cmyk = (0, 0, 100, 0)

print(vermelho_rgb[0])      255
print(amarelo_cmyk[2])     100
```

Tentou alterar? Erro.

```
vermelho_rgb[0] = 200
```

TypeError: 'tuple' object does not support item assignment

A pegadinha do 1 elemento

(5) → é um int

(5,) → é uma tupla

É a vírgula que cria a tupla.

Desempacotamento

Um passo elegante: tupla entra, variáveis nomeadas saem

Sem desempacotar

```
r = vermelho_rgb[0]
g = vermelho_rgb[1]
b = vermelho_rgb[2]
```

Desempacotando

```
r, g, b = vermelho_rgb
print(f"R: {r}, G: {g}, B: {b}")
R: 255, G: 0, B: 0
```

A quantidade tem que bater exatamente

`x, y = vermelho_rgb` → `ValueError: too many values to unpack (expected 2)`. Três valores não cabem em duas variáveis.

Tupla também aceita fatiamento: `vermelho_rgb[:2]` → `(255, 0)`. Tudo do Capítulo 3 continua valendo aqui.

Dicionários

- Dado com RÓTULO, não com posição: `pessoa["nome"]`
- A estrutura por trás de JSON, XML e praticamente toda API
- Mutável — e isso tem consequências dentro de funções

Dicionário: chave e valor

Chaves {} e o par chave: valor separado por vírgula

```

pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com"
}

print(pessoa["nome"])      João da Silva

pessoa["idade"] = 36       # altera
pessoa["cidade"] = "SP"   # cria nova chave
```

Removendo

```

del pessoa["telefone"]
    apaga

email = pessoa.pop("email")
    apaga E devolve o valor
```

Aninhamento

Um valor pode ser outro dicionário. Acesse em profundidade, um colchete por nível:

```
pessoa["endereco"]["cidade"]
```

Percorrer, consultar e não quebrar

items(), keys(), values(), in e get()

```
for chave, valor in pessoa.items():  
    print(f"{chave}: {valor}")  
  
print(pessoa.keys())    todas as chaves  
print(pessoa.values())  todos os valores
```

KeyError: o erro nº 1 com dicionários

```
pessoa["cpf"]           KeyError: 'cpf'  
  
pessoa.get("cpf")       None  
  
pessoa.get("cpf", "não informado")  
                        não informado
```

Atenção: erro no exemplo 10 do livro

O livro (p. 10) traz:

```
if "email" in pessoa:  
    print("E-mail cadastrado:",  
          pessoa["contato"]["email"])
```

Se digitado assim, quebra: `KeyError: 'contato'` — o dicionário `pessoa` não tem essa chave. A forma correta:

```
if "email" in pessoa:  
    print("E-mail cadastrado:",  
          pessoa["email"])
```

Tuplas e dicionários em funções

A diferença que mais gera bug: o dicionário vai por referência

Tupla: entra, sai, não muda

```
def calcular_idade_futura(atual, anos):  
    return atual, atual + anos  
  
a, f = calcular_idade_futura(35, 5)  
print(a, f)           35 40  
  
# o return devolve uma TUPLA
```

Dicionário: a função altera o original

```
def adicionar_email(p):  
    p["email"] = "joao@email.com"  
  
dados = {"nome": "João", "idade": 35}  
adicionar_email(dados)  
print(dados)  
  
# o dicionário FORA da função mudou
```

A solução: copy()

nova = p.copy() dentro da função, altere a cópia e devolva com return. O original fica intacto. Cuidado: copy() é raso — dicionários aninhados dentro dele continuam compartilhados (para esses, deepcopy).

3

PRÁTICA 3 · 4 minutos

Ficha de aluno

Arquivo: `pratica_03.py`

1. Crie um dicionário aluno com as chaves nome, curso e notas — sendo notas uma TUPLA de 3 valores
2. Imprima apenas a segunda nota
3. Escreva a função `situacao(aluno)` que calcula a média e ADICIONA a chave "situacao" com Aprovado (média ≥ 7) ou Reprovado. Depois imprima o dicionário.

Pistas

`aluno["notas"][1]` pega a 2ª nota

`média = sum(notas) / len(notas)`

No item 3 você NÃO precisa de `return`:
o dicionário vai por referência.

Pense: por que isso funciona?

O item 3 é o fecho da aula: ele só funciona por causa da passagem por referência.

Qual estrutura usar?

A pergunta que fica depois desta aula

Lista []

- Ordem importa
- Muda o tempo todo
- Itens do mesmo tipo
- Ex.: notas = [7.5, 6.0]

Tupla ()

- A ordem É o significado
- Não pode mudar
- Pode misturar tipos
- Ex.: (lat, long), RGB

Dicionário { }

- Rótulo importa, não posição
- Muda o tempo todo
- Valor pode ser qualquer coisa
- Ex.: {"nome": "Ana"}

E o fatiamento [início:fim:passo] funciona em lista, tupla e string — em tudo que é sequência.

PARA CASA

Boletim da turma

Integra tudo: lista, tupla, dicionário, função e fatiamento.

Ponto de partida

```
alunos = [  
    ("Ana", (8.0, 9.0, 10.0)),  
    ("Bruno", (5.0, 6.5, 7.0)),  
    ("Carla", (9.5, 9.0, 8.5)),  
    ("Diego", (4.0, 5.0, 6.0)),  
]
```

O que entregar

- Função boletim(alunos) devolvendo um dicionário por aluno
- Cada um com notas, media e situacao
- Imprimir só os 2 primeiros nomes em ordem alfabética
- Imprimir a média geral da turma

Confira: Bruno fica com média 6.17 e a média geral da turma é 7.29.

O que você leva desta aula

1

Fatiar

O fim é sempre excluído. O passo é o terceiro parâmetro.

2

Inverter

[::-1] resolve lista, tupla e string.

3

Escolher

Posição fixa → tupla. Rótulo → dicionário. Coleção que muda → lista.

4

Cuidar

Dicionário em função vai por referência. Use `copy()` quando não quiser alterar o original.